

File Storage & Sharing

Take-Home Assignment - Recommended Architecture & Implementation Blueprint

Goal: build a small, functional full-stack application for document storage, external sharing, workspaces, invitations, and access control - optimized for a weekend scope.

1. Recommended Stack

Layer	Choice	Why
Frontend	Next.js + TypeScript + Tailwind	Fast to build, functional UI, strong TypeScript support.
Backend	Node.js + TypeScript + Fastify	Lightweight REST API with good validation and structure.
Database	PostgreSQL + pg (raw SQL)	Explicit queries, strong SQL control, no ORM complexity.
Migrations	node-pg-migrate or simple ordered SQL files	Meets the migration requirement without introducing an ORM.
File storage	MinIO locally via S3 API	Keeps file bytes outside PostgreSQL; easy to swap for AWS S3.
Auth	Email/password + server-side session cookies	Simple browser auth with HttpOnly cookies.
Testing	Vitest + API/integration tests	Focuses effort on authorization, sharing, and storage boundaries.
Infra	Docker Compose	One command brings up web, API, PostgreSQL, and MinIO.

2. High-Level Architecture

Browser -> Next.js UI -> Node.js API -> PostgreSQL (metadata) + MinIO (file bytes).

Keep the application as a modular monolith. Do not introduce microservices, Redis, Kafka, Elasticsearch, or Kubernetes for this assignment.

3. Core Product Decisions

Area	Decision
Workspace roles	OWNER and MEMBER only. Keep authorization easy to reason about.
Shared links	Read-only public links; optional expiration; creator can revoke. Recipient does not need an account.
Invitations	Invite by email. An invited person can create an account or sign in before accepting.
Development email	Expose the generated invitation link in development instead of integrating an email provider.
File limits	Example: 25 MB maximum; allow common document/image types. Document this in README.
Deletion	Soft-delete document metadata, then remove the underlying object; ensure deleted docs are not downloadable.

4. Suggested PostgreSQL Schema

Table	Key fields
users	id, email, password_hash, created_at
workspaces	id, name, created_by, created_at
workspace_members	workspace_id, user_id, role, created_at; unique(workspace_id,user_id)
documents	id, workspace_id, uploaded_by, filename, storage_key, mime_type, size, created_at, deleted_at
shares	id, document_id, token_hash, expires_at, created_by, created_at, revoked_at
invitations	id, workspace_id, email, token_hash, role, expires_at, accepted_at, created_by, created_at

Indexes: workspace_id on documents; unique workspace/user membership; token lookup indexes as appropriate. Use foreign keys and transactional updates where related rows must change together.

5. Storage Abstraction

Store only metadata in PostgreSQL. Put the actual file in MinIO/S3. Hide the provider behind a small interface so infrastructure can change without rewriting business logic.

```
interface FileStorage { upload(key, stream, contentType); download(key); delete(key); getSignedUrl(key, expiresIn); }
```

Upload flow: validate user -> validate workspace -> validate file -> upload object -> persist metadata. **Download flow:** authorize -> verify not deleted -> return a short-lived signed URL/stream.

6. API Shape

Method	Route	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Create session
POST	/api/auth/logout	Destroy session
GET	/api/auth/me	Current user
POST	/api/workspaces	Create workspace
GET	/api/workspaces	List user workspaces
GET	/api/workspaces/:id/members	List members
POST	/api/workspaces/:id/invitations	Invite member
POST	/api/workspaces/:id/documents	Upload document
GET	/api/workspaces/:id/documents	List documents
DELETE	/api/documents/:id	Delete document
POST	/api/shares	Create share link
DELETE	/api/shares/:id	Revoke share link
GET	/api/shares/:token	Resolve/download shared file

7. Security & Authorization

Use password hashing (e.g. Argon2/bcrypt), HttpOnly + Secure cookies in production, server-side authorization on every workspace/document action, parameterized SQL, upload size/type validation, and opaque random share/invitation tokens. Prefer storing token hashes rather than raw tokens. Never make the MinIO bucket public.

8. Tests That Matter

Area	Examples
Authorization	Non-member cannot access another workspace or its documents.
Sharing	Valid link works; expired/revoked/invalid links fail.
Uploads	Valid upload succeeds; oversize/unsupported files are rejected.
Membership	Duplicate membership is prevented; invitation acceptance creates one membership.
Storage	Metadata is not persisted when an object upload fails (and cleanup is attempted on partial failure).

9. Docker Compose

Services: postgres, minio, api, web. The README should make `docker compose up` - build the primary startup path. Include environment variables, health checks where useful, migration execution, demo credentials or seed data, and URLs for the UI/API/MinIO console.

10. Keep the Weekend Scope Tight

Do not build OAuth, SSO, MFA, billing, comments, document collaboration, versioning, notifications, search infrastructure, or a complex design system. Functional UI and correct authorization matter more than polish.

11. Coding Agent / README Notes

The assignment explicitly asks you to use a coding agent. In the README, record the agent(s) used (for example Claude Code, Cursor, Codex, Copilot, or Aider), what you delegated, and what you personally reviewed, changed, or rejected. Emphasize architecture decisions and trade-offs rather than pretending the agent wrote everything without review.

README sections to include: Setup -> Architecture -> Product assumptions -> API overview -> Data model -> Storage abstraction -> Security decisions -> Test strategy -> Agent usage -> Trade-offs / future improvements.

12. Suggested Project Layout

```
apps/web/ apps/api/ src/db/ src/modules/auth/ src/modules/workspaces/ src/modules/documents/ src/modules/shares/
src/storage/ migrations/ tests/ docker-compose.yml README.md
```